# Trabajo Práctico Final

# Análisis de Lenguajes de Programación

*Simulador de Circuitos Cuánticos en Haskell*

---

**Descripción:** Desarrollo de un Lenguaje de Dominio Específico Embebido (**EDSL**) en Haskell para la simulación de circuitos cuánticos.

---


**Alumno:** Ignacio Rímini
**Materia:** Análisis de Lenguajes de Programación (ALP)
**Fecha:** Marzo 2026

Universidad Nacional de Rosario

# Índice

**1. Computación Cuántica. 3**
1.1. ¿Qué es la computación cuántica? . . . 3
1.2. El Qubit. . . . 3
1.3. Sistemas de múltiples qubits y producto tensorial. . . . 4
1.3.1. El producto tensorial. . . . 4
1.3.2. Vectores de estado en sistemas múltiples. . . . 4
1.4. Compuertas cuánticas. . . . 5
1.4.1. Compuerta Pauli-X (NOT). . . . 5
1.4.2. Compuerta Hadamard (H). . . . 5
1.4.3. Compuerta CNOT (Controlled-NOT). . . . 6
1.4.4. Compuerta CCNOT (Toffoli). . . . 6
1.5. Circuitos cuánticos. . . . 7
1.6. ¿Cómo se computa en la computación cuántica? . . . 7

**2. Diseño del Lenguaje de Dominio Específico Embebido (EDSL). 9**
2.1. Sintaxis concreta. Lenguaje .q . . . 9
2.2. Gramática Formal (EBNF). . . . 9
2.2.1. Precedencia y Asociatividad . . . 10

**3. Arquitectura del Sistema. 11**
3.1. Estructura de Archivos. . . . 11
3.2. Módulo `AST.hs`: Sintaxis Abstracta. . . . 11
3.3. Módulo `Parser.hs`: Análisis Sintáctico con Parsec. . . . 11

**4. El Evaluador: La Mónada QState. 13**
4.1. Definición de la mónada. . . . 13
4.1.1. Instancia de Monad y operadores. . . . 13
4.2. Verificación de las leyes monádicas. . . . 14
4.2.1. Demostración de Monad.1. . . . 14
4.2.2. Demostración de Monad.2. . . . 14
4.2.3. Demostración de Monad.3. . . . 15
4.3. Operaciones Primitivas. . . . 15
4.4. Manejo de errores y validaciones. . . . 16
4.5. Álgebra lineal cuántica. . . . 16
4.5.1. Operaciones base de álgebra lineal. . . . 16
4.5.2. Expansión de compuertas al espacio global. . . . 17
4.5.3. Compuertas controladas: el problema de los SWAPs. . . . 17
4.5.4. Expansión de CNOT y CCNOT por manipulación de bits. . . . 17
4.6. Validación y seguridad del circuito. . . . 18
4.6.1. Validación de rangos (`checkQubit`). . . . 18
4.6.2. Análisis de qubits usados (`usedQubits`). . . . 19
4.7. Denotación matemática de compuertas (`gateToMatrix`). . . . 19
4.8. Ejecución y flujo principal: La función `eval`. . . . 20
4.8.1. Aplicación del circuito (`applyCircuit`). . . . 20
4.8.2. Punto de entrada: `eval`. . . . 20

**5. Módulo `PrettyPrinter.hs`. 21**

5.1. Conversión a binario (`intToBinary`). . . . . 21
5.2. Formateo de amplitudes (`formatComplex`). . . . . 21
5.3. Representación de Dirac (`prettyPrint`). . . . . 21

**6. Compilación y guía de uso. 23**
6.1. Compilación del sistema. . . . . 23
6.2. Ejecución de circuitos. . . . . 23
6.2.1. Ejecución por defecto. . . . . 23
6.2.2. Ejecución de archivos específicos. . . . . 23

**7. Ejemplos y Resultados. 24**
7.1. Estado de Bell (`bell.q`). . . . . 24
7.2. Estado GHZ de 3 qubits (`ghz.q`). . . . . 24
7.3. Algoritmo de Búsqueda de Grover (`grover.q`). . . . . 25
7.3.1. Propósito y fundamento. . . . . 25
7.3.2. Implementación en 2 qubits. . . . . 25
7.3.3. Resultado. . . . . 26
7.4. Medio sumador cuántico (`adder.q`). . . . . 26

**8. Decisiones de Diseño y Justificaciones. 28**

**9. Posibles extensiones futuras. 28**

**10. Conclusiones. 29**

# 1. Computación Cuántica.

## 1.1. ¿Qué es la computación cuántica?

La **computación cuántica** es un paradigma de procesamiento de información que se basa en los principios de la mecánica cuántica para realizar cálculos. A diferencia de la computación clásica, que utiliza bits como unidad fundamental (siempre en un estado definido de 0 o 1), la computación cuántica utiliza **qubits**.

Este paradigma no es simplemente una evolución de la computación clásica, sino un cambio fundamental en la naturaleza misma del cómputo. Al aprovechar fenómenos como la **superposición**, el **entrelazamiento** y la **interferencia**, es posible diseñar algoritmos que resuelven problemas complejos —como la factorización de números grandes o la simulación de moléculas— en tiempos significativamente menores que los mejores algoritmos clásicos conocidos.

## 1.2. El Qubit.

El **qubit (quantum bit)** es el bloque básico de información cuántica. Mientras que un bit clásico solo puede estar en el estado 0 o en el 1, un qubit puede existir en una combinación lineal de ambos estados:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$$

Matemáticamente, un qubit se representa como un vector unitario en un **espacio de Hilbert** complejo de dos dimensiones ($\mathbb{C}^2$). Utilizando la **notación de Dirac** (Bra-Ket), el estado de un qubit $|\psi\rangle$ se describe como una combinación lineal de los estados base computacionales $|0\rangle$ y $|1\rangle$:

$$|0\rangle = \begin{pmatrix}1\\0\end{pmatrix}, \quad |1\rangle = \begin{pmatrix}0\\1\end{pmatrix}$$

donde $\alpha, \beta \in \mathbb{C}$ son las *amplitudes de probabilidad*. Estos valores deben cumplir la condición de **normalización**:

$$|\alpha|^2 + |\beta|^2 = 1$$

Físicamente, esta condición asegura que la probabilidad total de encontrar el sistema en alguno de sus estados posibles al realizar una medición sea exactamente igual a 1. Esto implica que el cuadrado del módulo de la amplitud ($|\alpha|^2$ o $|\beta|^2$) representa la probabilidad de colapso al estado base correspondiente.

**Observaciones.**

- **Espacio de Hilbert:** Es un espacio vectorial complejo dotado de un producto interno. En el contexto de un solo qubit, este espacio es $\mathbb{C}^2$.
- **Notación de Dirac:** Introducida por Paul Dirac, esta representación simplifica el álgebra lineal cuántica utilizando "kets"($|\psi\rangle$) para vectores columna y "bras"($\langle\psi|$) para sus adjuntos hermíticos. El producto interno se representa como $\langle\phi|\psi\rangle$.

## 1.3. Sistemas de múltiples qubits y producto tensorial.

Un sistema con $n$ qubits no vive en una suma de espacios, sino en su **producto tensorial** ($\otimes$). El espacio de estados global para $n$ qubits es $(\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$.

Esta dimensión crece de forma exponencial: un sistema de solo 10 qubits requiere un vector de 1024 componentes complejas para su descripción. La base de un sistema de dos qubits, por ejemplo, está formada por los vectores $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$, y cualquier estado general es una superposición de estos cuatro estados base.

### 1.3.1. El producto tensorial.

El **producto tensorial** es una operación que combina dos espacios vectoriales para formar uno nuevo de mayor dimensión. Si tenemos dos qubits individuales en los estados $|\psi\rangle$ y $|\phi\rangle$, el estado conjunto del sistema es:

$$|\Psi\rangle = |\psi\rangle \otimes |\phi\rangle$$

Matemáticamente, si $|\psi\rangle = \begin{pmatrix} a \\ b \end{pmatrix}$ y $|\phi\rangle = \begin{pmatrix} c \\ d \end{pmatrix}$, su producto tensorial se calcula como:

$$|\psi\rangle \otimes |\phi\rangle = \begin{pmatrix} a\begin{pmatrix} c \\ d \end{pmatrix} \\ b\begin{pmatrix} c \\ d \end{pmatrix} \end{pmatrix} = \begin{pmatrix} ac \\ ad \\ bc \\ bd \end{pmatrix}$$

Este nuevo vector de 4 dimensiones representa todas las combinaciones posibles de los estados base de cada qubit individual ($|00\rangle, |01\rangle, |10\rangle, |11\rangle$).

### 1.3.2. Vectores de estado en sistemas múltiples.

Un sistema de $n$ qubits se describe mediante un **único vector de estado global** en el espacio complejo $\mathbb{C}^{2^n}$. Es fundamental entender que, sin importar cuántos qubits compongan el sistema, el estado total se representa con un solo vector columna, cuya dimensión es la que crece exponencialmente ($2^n$).

Este vector contiene todas las amplitudes de probabilidad del sistema:

$$|\Psi\rangle = \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_{2^n-1} \end{pmatrix} = \sum_{i=0}^{2^n-1} c_i \, |i\rangle_n = c_0 \, |00\ldots0\rangle + c_1 \, |00\ldots1\rangle + \cdots + c_{2^n-1} \, |11\ldots1\rangle$$

Donde:

- $c_i \in \mathbb{C}$ es la amplitud de probabilidad del estado $i$.
- $|i\rangle_n$ representa la cadena binaria de longitud $n$ correspondiente al índice $i$.
- Se debe seguir cumpliendo la condición de normalización global: $\sum |c_i|^2 = 1$.

Por ejemplo, un sistema de 3 qubits se describe con un *único vector de 8 dimensiones.* Cada una de estas 8 componentes complejas ($c_i$) está asociada a una configuración binaria específica de los qubits ($|000\rangle, |001\rangle, \ldots$), lo cual se visualiza en el siguiente vector columna:

$$|\Psi\rangle_{3\text{qubits}} = \begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \\ c_6 \\ c_7 \end{pmatrix} \begin{matrix} \leftarrow \text{Amplitud de } |000\rangle \\ \leftarrow \text{Amplitud de } |001\rangle \\ \leftarrow \text{Amplitud de } |010\rangle \\ \leftarrow \text{Amplitud de } |011\rangle \\ \leftarrow \text{Amplitud de } |100\rangle \\ \leftarrow \text{Amplitud de } |101\rangle \\ \leftarrow \text{Amplitud de } |110\rangle \\ \leftarrow \text{Amplitud de } |111\rangle \end{matrix}$$

## 1.4. Compuertas cuánticas.

Las operaciones que modifican el estado de los qubits se denominan **compuertas cuánticas**. Estas existen para permitir la manipulación controlada de la información cuántica, funcionando como los componentes básicos que permiten construir algoritmos.

A diferencia de las compuertas lógicas clásicas (como AND u OR), las compuertas cuánticas son **operadores lineales reversibles**. Matemáticamente, se representan mediante **matrices unitarias** ($U$), las cuales deben cumplir que $U^\dagger U = I$. Esta propiedad garantiza que la norma del vector de estado se conserve siempre en 1, asegurando que las probabilidades sumen 1 en todo momento del cómputo.

Proyectar una compuerta sobre un estado $|\psi\rangle$ consiste en realizar el producto matriz-vector: $|\psi'\rangle = U \cdot |\psi\rangle$.

### 1.4.1. Compuerta Pauli-X (NOT).

Es el análogo cuántico de la compuerta NOT clásica. Su función es realizar un *bit-flip*, intercambiando las amplitudes de los estados $|0\rangle$ y $|1\rangle$. Se representa mediante la siguiente matriz:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Si el qubit está en el estado $|0\rangle$, la aplicación de la compuerta resulta en:

$$X\,|0\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} = |1\rangle$$

### 1.4.2. Compuerta Hadamard (H).

Es una de las compuertas más fundamentales, ya que es la encargada de crear **superposición**. Transforma los estados base en combinaciones lineales donde ambos resultados tienen la misma probabilidad de ser medidos. Se representa mediante la siguiente matriz:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Cuando se aplica a $|0\rangle$, produce el estado $|+\rangle$:

$$H\,|0\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}\begin{pmatrix}1\\ 0\end{pmatrix} = \frac{1}{\sqrt{2}}\begin{pmatrix}1\\ 1\end{pmatrix} = \frac{1}{\sqrt{2}}\,|0\rangle + \frac{1}{\sqrt{2}}\,|1\rangle$$

### 1.4.3. Compuerta CNOT (Controlled-NOT).

Es una compuerta de dos qubits esencial para generar **entrelazamiento**. Posee un qubit de *control* y un qubit *objetivo*. La operación invierte el estado del objetivo solo si el control está en el estado $|1\rangle$. Se representa mediante la siguiente matriz:

$$CNOT = \begin{pmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 1\\ 0 & 0 & 1 & 0\end{pmatrix}$$

Si la aplicamos al estado $|10\rangle$ (donde el control es 1 y el objetivo 0):

$$\text{CNOT}\,|10\rangle = \begin{pmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 1\\ 0 & 0 & 1 & 0\end{pmatrix}\begin{pmatrix}0\\ 0\\ 1\\ 0\end{pmatrix} = \begin{pmatrix}0\\ 0\\ 0\\ 1\end{pmatrix} = |11\rangle$$

### 1.4.4. Compuerta CCNOT (Toffoli).

También conocida como **compuerta de Toffoli,** es una compuerta de tres qubits con dos controles y un objetivo. El objetivo se invierte solo si ambos controles son $|1\rangle$. Se representa mediante la siguiente matriz:

$$CCNOT = \begin{pmatrix}1 & 0 & 0 & 0 & 0 & 0 & 0 & 0\\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0\\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0\\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1\\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0\end{pmatrix}$$

Si la aplicamos al estado $|110\rangle$ (ambos controles en 1, objetivo en 0):

$$\text{CCNOT}\,|110\rangle = \begin{pmatrix}1 & \dots & 0\\ \vdots & \ddots & \vdots\\ 0 & \dots & 1\end{pmatrix}\begin{pmatrix}0\\ 0\\ 0\\ 0\\ 0\\ 0\\ 1\\ 0\end{pmatrix} = \begin{pmatrix}0\\ 0\\ 0\\ 0\\ 0\\ 0\\ 0\\ 1\end{pmatrix} = |111\rangle$$

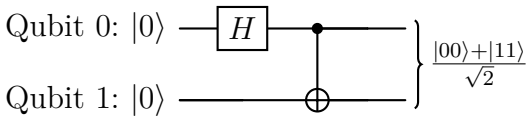
## 1.5. Circuitos cuánticos.

Un **circuito cuántico** es la representación gráfica de un algoritmo cuántico. A diferencia de los diagramas de flujo clásicos, estos se leen estrictamente de **izquierda a derecha**, donde cada línea horizontal representa la 'línea de vida' de un qubit.

Para interpretar estos gráficos, debemos entender el significado de sus componentes visuales:

- **Líneas horizontales (Cables):** Cada línea representa un qubit individual. Mientras la línea sea continua y vacía, el qubit mantiene su estado actual a través del tiempo.
- **Cajas con letras:** Representan la aplicación de una compuerta sobre ese qubit específico. Por ejemplo, una caja con una 'H' indica que se aplica una transformación de Hadamard, mientras que una 'X' indica una inversión de estado (NOT).
- **Puntos sólidos y líneas verticales:** Indican una conexión de **control**. Un punto sobre una línea significa que la operación de esa columna solo se ejecutará si ese qubit específico se encuentra en el estado $|1\rangle$.
- **Círculos con una cruz ($\oplus$):** Representan el **objetivo** de una operación controlada (como la CNOT). Es el lugar donde se aplica la inversión de estado si el control lo permite.

**Ejemplo: El circuito de Bell.**




En este diagrama:

1. Ambos qubits inician en $|0\rangle$.
2. Se aplica una compuerta **H** al Qubit 0, poniéndolo en superposición.
3. La línea vertical conecta un **control** en el Qubit 0 con un **objetivo** en el Qubit 1. Esto es una compuerta CNOT.
4. El resultado final (a la derecha) es el estado entrelazado de Bell.

## 1.6. ¿Cómo se computa en la computación cuántica?

El **proceso de cómputo** en un sistema cuántico no consiste en mover bits por cables, sino en la **manipulación de un vector de estado mediante álgebra lineal**. El proceso sigue dos etapas fundamentales:

1. **Preparación (Inicialización):** Se define el vector de estado inicial. Para $n$ qubits, es un vector de dimensión $2^n$ con un 1 en la primera posición y 0 en las demás, representando el estado $|00\ldots0\rangle$.

2. **Evolución (Aplicación de compuertas):** Cada paso del cronograma es matemáticamente una multiplicación de matrices. Si el circuito tiene las compuertas $G_1, G_2, \ldots, G_k$, el simulador calcula:

$$|\psi_{final}\rangle = G_k \cdot (\cdots (G_2 \cdot (G_1 \cdot |\psi_{inicial}\rangle)))$$

Al finalizar las operaciones, el 'resultado' es el vector de estado final.

# 2. Diseño del Lenguaje de Dominio Específico Embebido (EDSL).

Este trabajo implementa un **EDSL** (*Embedded Domain-Specific Language*) en Haskell para la descripción y simulación de circuitos cuánticos. Se optó por una estrategia de **Deep Embedding**, donde las construcciones del lenguaje se representan como tipos de datos algebraicos (AST), permitiendo múltiples interpretaciones como la simulación, la optimización o la inspección visual del circuito antes de su ejecución.

Complementando el núcleo en Haskell, el sistema provee una **sintaxis concreta externa** mediante archivos con extensión `.q`. Esta interfaz textual permite definir circuitos de forma imperativa y es procesada por un analizador sintáctico (*parser*) que traduce el código fuente en la estructura de datos interna del EDSL.

## 2.1. Sintaxis concreta. Lenguaje .q

Los circuitos se describen en archivos de texto con extensión `.q`. Cada archivo debe comenzar con la palabra clave `qubits` seguida de un número entero que define el tamaño del sistema. A continuación, se definen las compuertas utilizando los siguientes operadores de composición:

- **Secuencial (`;`)**: Indica que las compuertas se ejecutan una después de la otra en el tiempo.
- **Paralelo (`||`)**: Permite aplicar compuertas simultáneamente sobre qubits distintos. Por ejemplo, `H 0 || X 1` aplica una Hadamard al qubit 0 y un NOT al qubit 1 al mismo tiempo.
- **Paréntesis (`()`)**: Se utilizan para agrupar expresiones y alterar la precedencia por defecto de los operadores.

```
// Circuito de Bell (2 qubits)
qubits 2
H 1 ; CNOT 1 0
```

*Ejemplo de sintaxis: Estado de Bell (`bell.q`)*

```
// Circuito GHZ (3 qubits)
qubits 3
H 2 ; CNOT 2 1 ; CNOT 1 0
```

*Ejemplo de sintaxis: Estado GHZ de 3 qubits (`ghz.q`)*

## 2.2. Gramática Formal (EBNF).

La gramática del lenguaje de circuitos es una **Gramática Libre de Contexto (CFG)** definida mediante la siguiente notación EBNF:

$$
\begin{array}{rcl}
\langle \text{circuit} \rangle & ::= & \textbf{qubits}\ \langle \text{number} \rangle\ \langle \text{gate_expr} \rangle \\
\langle \text{gate_expr} \rangle & ::= & \langle \text{gate_term} \rangle\ \{\ \texttt{';'}\ \langle \text{gate_term} \rangle\ \} \\
\langle \text{gate_term} \rangle & ::= & \langle \text{gate_factor} \rangle\ \{\ \texttt{'||'}\ \langle \text{gate_factor} \rangle\ \} \\
\langle \text{gate_factor} \rangle & ::= & \textbf{H}\ \langle \text{number} \rangle \\
 & | & \textbf{X}\ \langle \text{number} \rangle \\
 & | & \textbf{CNOT}\ \langle \text{number} \rangle\ \langle \text{number} \rangle \\
 & | & \textbf{CCNOT}\ \langle \text{number} \rangle\ \langle \text{number} \rangle\ \langle \text{number} \rangle \\
 & | & \texttt{'('}\ \langle \text{gate_expr} \rangle\ \texttt{')'} \\
\langle \text{number} \rangle & ::= & [0\text{-}9]+
\end{array}
$$

### 2.2.1. Precedencia y Asociatividad

Para evitar ambigüedades en expresiones complejas, se han definido las siguientes reglas:

- **Precedencia:** El operador de paralelismo (`||`) tiene mayor precedencia que el operador de secuencia (`;`). Esto significa que `H 0 || X 1 ; CNOT 1 0` se interpreta como aplicar primero las dos compuertas en paralelo y luego la CNOT.
- **Asociatividad:** Ambos operadores asocian hacia la izquierda, permitiendo cadenas largas como `g1 ; g2 ; g3` de forma natural.

# 3. Arquitectura del Sistema.

## 3.1. Estructura de Archivos.

La organización del código fuente se divide en tres directorios principales:

```
tp-final-alp/
 app/
    Main.hs           -- Punto de entrada del ejecutable
 src/
    AST.hs            -- Sintaxis Abstracta
    Eval.hs           -- Evaluador Semantico
    Examples.hs       -- Ejemplos embebidos
    Parser.hs         -- Analizador Sintactico
    PrettyPrinter.hs  -- Impresion en notacion Dirac
 examples-q/
     adder.q           -- Sumador de 4 qubits
     bell.q            -- Estado entrelazado 2 qubits
     ghz.q             -- Estado GHZ 3 qubits
     grover.q          -- Algoritmo Grover
     error-parallel.q  -- Test: error en paralelo
     error-qubit.q     -- Test: qubit fuera de rango
     error-syntax.q    -- Test: error de sintaxis
```

## 3.2. Módulo `AST.hs`: Sintaxis Abstracta.

Este módulo define el núcleo del EDSL mediante el uso de **GADTs** (*Generalized Algebraic Data Types*):

```haskell
data Gate where
   H     :: Qubit -> Gate                          -- Hadamard
   X     :: Qubit -> Gate                          -- Pauli-X (NOT)
   CNOT  :: Qubit -> Qubit -> Gate                 -- Controlled-NOT
   CCNOT :: Qubit -> Qubit -> Qubit -> Gate        -- Toffoli
   Seq   :: Gate -> Gate -> Gate                   -- Secuencia (;)
   Par   :: Gate -> Gate -> Gate                   -- Composicion
      paralela (||)
```

Esta estructura permite capturar la jerarquía de operadores definida en la gramática. El uso de un tipo de datos dedicado facilita que el evaluador pueda recorrer el árbol de compuertas y transformarlo en las operaciones matriciales finales.

## 3.3. Módulo `Parser.hs`: Análisis Sintáctico con Parsec.

El analizador sintáctico transforma el código textual de los archivos `.q` en el tipo de datos `Gate` definido en el AST. Se utiliza la librería **Parsec**, un combinador de *parsers* monádico que permite una definición cercana a la gramática formal:

- **Consumo de Tokens:** Se encargan de procesar identificadores, números enteros y operadores reservados, ignorando espacios en blanco y comentarios automáticamente.
- **Composición de Parsers:** Las producciones de la gramática se delegan a funciones específicas que manejan la precedencia. Por ejemplo, el operador `||` tiene mayor peso que el `;` mediante el uso de `chainl1`, lo que garantiza que las expresiones se agrupen correctamente.
- **Manejo de Errores:** Si el archivo contiene errores de sintaxis (como un número incorrecto de qubits o un operador mal escrito), Parsec genera mensajes detallados indicando la fila y columna exacta del problema.

```haskell
-- Secuencia (asocia a izquierda).
-- <gate_expr> ::= <gate_term> { ';' <gate_term> }
gateExpr :: Parser Gate
gateExpr = chainl1 gateTerm (reservedOp ";" >> return Seq)

-- Paralelo (asocia a izquierda).
-- <gate_term> ::= <gate_factor> { '||' <gate_factor> }
gateTerm :: Parser Gate
gateTerm = chainl1 gateFactor (reservedOp "||" >> return Par)

-- Atomos y parentesis.
gateFactor :: Parser Gate
gateFactor = parens gateExpr
            <|> (reserved "H"  >> H  <$> natural)
            <|> (reserved "X"  >> X  <$> natural)
            <|> (reserved "CNOT" >> CNOT <$> natural <*> natural)
            <|> (reserved "CCNOT" >> CCNOT <$> natural <*>
               natural <*> natural)
```

*Parser.hs - Combinadores principales.*

# 4. El Evaluador: La Mónada QState.

El corazón del simulador es la mónada `QState`, un tipo de datos diseñado específicamente para encapsular la computación cuántica, permitiendo el manejo de estados mutables (vectores columna), la propagación de errores y la acumulación de trazas de ejecución de forma puramente funcional.

## 4.1. Definición de la mónada.

Un programa cuántico necesita mantener un estado (el vector de amplitudes) que cambia con cada operación, y ser capaz de fallar si se aplican operaciones inválidas. La mónada `QState` abstrae este comportamiento:

```
newtype QState a = QState {
    runQState :: StateVector -> Either QError (a, StateVector,
       QTrace)
}
```

*Definición de la mónada QState.*

Esta definición combina las capacidades de tres mónadas clásicas:

- **State**: Para propagar el `StateVector`.
- **Error**: Para el manejo de excepciones mediante el tipo `QError`.
- **Writer**: Para acumular los mensajes del log de ejecución (`QTrace`).

### 4.1.1. Instancia de Monad y operadores.

Para que `QState` se comporte como una mónada, debemos definir explícitamente cómo se inyectan valores puros y cómo se encadenan las operaciones. La implementación de las instancias es la siguiente:

```
instance Monad QState where
    return x = QState (\s -> Right (x, s, []))

    m >>= f = QState (\s ->
        case runQState m s of
            Left e1 -> Left e1
            Right (v1, s', t1) ->
                case runQState (f v1) s' of
                    Left e2 -> Left e2
                    Right (v2, s'', t2) -> Right (v2, s'', t1 ++
                       t2))
```

*Instancia de Monad para QState.*

Como se observa, el operador *bind* realiza tres tareas críticas:

1. **Cómputo secuencial:** Pasa el resultado $s'$ de la primera operación a la segunda.

2. **Gestión de errores:** Si la primera operación devuelve `Left`, la segunda no se ejecuta (cortocircuito).
3. **Acumulación:** Concatena los logs de la primera parte con los de la segunda (`t1 ++ t2`).

## 4.2. Verificación de las leyes monádicas.

Para garantizar que `QState` se comporta correctamente como una estructura monádica, debemos verificar que su implementación satisface las tres leyes fundamentales.

- **Monad.1 (Identidad Izquierda):** `return a >>= k = k a`
- **Monad.2 (Identidad Derecha):** `m >>= return = m`
- **Monad.3 (Asociatividad):** `m >>= (\x -> k x >>= h) = (m >>= k) >>= h`

### 4.2.1. Demostración de Monad.1.

$$
\begin{aligned}
& \mathit{return}\ a \gg= k \\
&= \mathit{QState}\ (\lambda s \to \text{case runQState } (\mathit{return}\ a)\ s \text{ of} \ldots) && \{\text{def } \gg= \} \\
&= \mathit{QState}\ (\lambda s \to \text{case Right } (a, s, []) \text{ of} && \{\text{def return, runQState}\} \\
&\quad \text{Right } (v, s', t) \to \text{runQState } (k\ v)\ s' \text{ con traza } [] \mathbin{++} t) \\
&= \mathit{QState}\ (\lambda s \to \text{runQState } (k\ a)\ s) && \{\text{pattern matching, } [] \text{ neutro}\} \\
&= \mathit{QState}\ (\text{runQState } (k\ a)) && \{\eta\text{-expansión}\} \\
&= k\ a && \{\text{def QState/runState}\}
\end{aligned}
$$

### 4.2.2. Demostración de Monad.2.

$$
\begin{aligned}
& m \gg= \mathit{return} \\
&= \mathit{QState}\ (\lambda s \to \text{case runQState } m\ s \text{ of} && \{\text{def } \gg= \} \\
&\quad \text{Right } (v, s', t) \to \text{runQState } (\mathit{return}\ v)\ s' \text{ con } t \mathbin{++} []) \\
&= \mathit{QState}\ (\lambda s \to \text{case runQState } m\ s \text{ of} && \{\text{def return, runQState}\} \\
&\quad \text{Right } (v, s', t) \to \text{Right } (v, s', t)) \\
&= \mathit{QState}\ (\lambda s \to \text{runQState } m\ s) && \{\text{id. en case y } [] \text{ neutro}\} \\
&= m && \{\eta\text{-exp., def QState}\}
\end{aligned}
$$

### 4.2.3. Demostración de Monad.3.

Para la asociatividad, demostraremos que ambos lados reducen a la misma forma funcional:

$$
\begin{aligned}
&(m \gg= k) \gg= h \\
&= QState\ (\lambda s \to \text{case runQState } (m \gg= k)\ s \text{ of} && \{\text{def } \gg= \} \\
&\quad \text{Right } (v, s', t) \to \text{runQState } (h\ v)\ s') \\
&= QState\ (\lambda s \to \text{case (case runQState } m\ s \text{ of} && \{\text{expandir } m \gg= k\} \\
&\quad \text{Right } (x, s_x, t_x) \to \text{runQState } (k\ x)\ s_x) \text{ of} \\
&\quad \text{Right } (y, s_y, t_y) \to \text{runQState } (h\ y)\ s_y) \\
&= QState\ (\lambda s \to \text{case runQState } m\ s \text{ of} && \{\text{aislar } m \text{ y aplanar case}\} \\
&\quad \text{Right } (x, s_x, t_x) \to \text{case runQState } (k\ x)\ s_x \text{ of} \\
&\qquad \text{Right } (y, s_y, t_y) \to \text{runQState } (h\ y)\ s_y)
\end{aligned}
$$

Partiendo del otro lado:

$$
\begin{aligned}
&m \gg= (\lambda x \to k\ x \gg= h) \\
&= QState\ (\lambda s \to \text{case runQState } m\ s \text{ of} && \{\text{def } \gg= \} \\
&\quad \text{Right } (x, s', t) \to \text{runQState } ((\lambda x \to k\ x \gg= h)\ x)\ s') \\
&= QState\ (\lambda s \to \text{case runQState } m\ s \text{ of} && \{\beta\text{-reducción}\} \\
&\quad \text{Right } (x, s', t) \to \text{runQState } (k\ x \gg= h)\ s') \\
&= QState\ (\lambda s \to \text{case runQState } m\ s \text{ of} && \{\text{def } \gg= \} \\
&\quad \text{Right } (x, s', t) \to \text{case runQState } (k\ x)\ s' \text{ of} \\
&\qquad \text{Right } (y, s'', t') \to \text{runQState } (h\ y)\ s'')
\end{aligned}
$$

Renombrando variables locales, ambas expresiones son equivalentes, demostrando que la composición monádica es asociativa y la propagación de errores y trazas es idéntica en ambos casos.

## 4.3. Operaciones Primitivas.

Para interactuar con el estado interno y la traza de la mónada, se han definido funciones primitivas que encapsulan el acceso a los datos de `QState`:

```
-- Obtiene el vector de estado actual para poder operar con el.
getState :: QState StateVector
getState = QState (\s -> Right (s, s, []))

-- Reemplaza el estado viejo por uno nuevo.
putState :: StateVector -> QState ()
putState s = QState (\_ -> Right ((), s, []))

-- Lanza un error y termina la computacion.
throwQError :: QError -> QState a
```

```
throwQError e = QState (\_ -> Left e)

-- Agrega un mensaje a la lista de la traza.
addTrace :: String -> QState ()
addTrace msg = QState (\s -> Right (() , s, [msg]))
```

## 4.4. Manejo de errores y validaciones.

El simulador implementa un sistema de validaciones robusto para garantizar que los circuitos descritos en los archivos `.q` sean físicamente posibles y computacionalmente viables:

- **`QubitOutOfBounds`**: Se activa si una compuerta intenta actuar sobre un índice de qubit que excede el tamaño definido en la cabecera del circuito.
- **`InvalidCircuit`**: Se utiliza para validar la estructura del circuito, especialmente para asegurar que las compuertas aplicadas en paralelo sean disjuntas y no compartan qubits.
- **`DimensionMismatch`**: Este error se produce si, durante las operaciones internas de álgebra lineal, el tamaño de una matriz de transformación no coincide con la dimensión del vector de estado global ($2^n$). Esto garantiza la integridad matemática del simulador.
- **`TooManyQubits`**: Impone un límite preventivo (20 qubits) para evitar que el crecimiento exponencial de la matriz de estado agote la memoria RAM del sistema.

## 4.5. Álgebra lineal cuántica.

### 4.5.1. Operaciones base de álgebra lineal.

Antes de aplicar la lógica cuántica, el sistema implementa las funciones básicas de manejo de vectores y matrices:

- **`initialState`**: Genera el vector de estado inicial $|0\rangle^{\otimes n}$. Es un vector de tamaño $2^n$ con un 1 en la primera posición.

```
initialState n = (1 :+ 0) : replicate (2^n - 1) (0 :+ 0)
```

- **`mvecmult`**: Realiza la multiplicación de una matriz de $2^n \times 2^n$ por un vector de $2^n$ componentes.

```
mvecmult m v = [ sum (zipWith (*) row v) | row <- m ]
```

- **`mmult`**: Implementa el producto de matrices complejo.

```
mmult a b = [ [ sum (zipWith (*) row col) | col <- transpose
   b ] | row <- a ]
```

- **`tensor`**: Computa el **Producto de Kronecker**, permitiendo combinar sistemas menores en espacios de mayor dimensión.

```
tensor a b = [ [x * y | x <- rowA, y <- rowB] | rowA <- a,
    rowB <- b]
```

### 4.5.2. Expansión de compuertas al espacio global.

Cuando se aplica una compuerta de un solo qubit a un sistema de $n$ qubits, es necesario proyectarla al espacio completo de dimensión $2^n$. Matemáticamente, una compuerta $U$ aplicada al qubit $q$ (donde $q$ es el índice del qubit) se expande realizando el producto tensorial con matrices identidad ($I$):

$$I^{\otimes(n-q-1)} \otimes U \otimes I^{\otimes q}$$

En el código, esto se implementa con `expandGate`:

```
expandGate :: Int -> Int -> Matrix -> Matrix
expandGate n q mat = foldr1 tensor (reverse [if i == q then mat
    else matI | i <- [0..n-1]])
```

El uso de `reverse` garantiza que el qubit 0 sea el bit menos significativo (LSB), consistente con la representación binaria.

### 4.5.3. Compuertas controladas: el problema de los SWAPs.

En el hardware cuántico real, las compuertas controladas (como CNOT) suelen aplicarse solo sobre qubits físicamente adyacentes. Cuando se requiere una CNOT entre qubits lejanos, se utiliza la técnica de **SWAPs**: se intercambia el estado del control con qubits vecinos hasta que este queda al lado del objetivo, se aplica la CNOT y luego se deshacen los intercambios.

Esta técnica presenta dos grandes desventajas para un simulador de software:

1. **Inestabilidad matemática:** Cada SWAP es una multiplicación de matrices. En sistemas grandes, el error acumulado y la complejidad de rastrear los índices hacen que el código sea propenso a fallos.
2. **Costo computacional:** Realizar múltiples productos de matrices de tamaño $2^n \times 2^n$ es extremadamente ineficiente, con un costo de $\mathcal{O}(N^3)$ por cada multiplicación.

### 4.5.4. Expansión de CNOT y CCNOT por manipulación de bits.

Para resolver esto, optamos por una **construcción directa de la matriz global** mediante la inspección de los bits de los índices del vector de estado. Esta técnica tiene un costo de $\mathcal{O}(N^2)$ y es mucho más elegante.

La lógica es la siguiente: para cada par de índices $(i, j)$ de la matriz global, determinamos si el elemento $(i, j)$ es 1 o 0 basándonos en la definición semántica de la compuerta.

```
expandCNOT n c t = [[cnotElement i j | j <- [0..limit]] | i <-
    [0..limit]]
    where
        limit = 2^n - 1
        cnotElement i j =
            if testBit i c  --  El  bit de control esta en 1?
                then if j == (i `xor` (1 `shiftL` t)) then 1 :+ 0
                    else 0 :+ 0
                else if i == j then 1 :+ 0 else 0 :+ 0
```

*Implementación de expandCNOT.*

**Análisis detallado:**

- **testBit i c**: Verifica si el qubit de control `c` está activo en el estado base `i`.
- **xor y shiftL**: Si el control está activo, el estado `i` debe transformarse en un estado `j` donde el bit en la posición `t` (objetivo) esté invertido. Esto se logra haciendo un XOR con $2^t$.
- **Identidad condicional**: Si el control es 0, la compuerta no hace nada, por lo que el elemento es 1 solo si $i = j$.

Esta técnica se extiende a la **CCNOT**, verificando dos bits de control simultáneamente:

```
ccnotElement i j =
    if testBit i c1 && testBit i c2  --  Ambos  controles en 1?
        then if j == (i `xor` (1 `shiftL` t)) then 1 :+ 0 else 0
            :+ 0
        else if i == j then 1 :+ 0 else 0 :+ 0
```

## 4.6. Validación y seguridad del circuito.

Antes de realizar el cómputo, el sistema debe garantizar que el circuito es físicamente consistente. Para esto, se implementan dos mecanismos de validación:

### 4.6.1. Validación de rangos (`checkQubit`).

Esta función asegura que cada compuerta actúe sobre un índice de qubit que exista en el sistema definido. De esta manera, se evita que el simulador intente acceder a posiciones de memoria inexistentes en el vector de estado.

```
checkQubit :: Int -> Int -> QState ()
checkQubit n q =
    if q >= 0 && q < n
    then return ()
    else throwQError (QubitOutOfBounds q)
```

### 4.6.2. Análisis de qubits usados (`usedQubits`).

En computación cuántica, dos operaciones que ocurren en paralelo no pueden compartir el mismo qubit objetivo ni de control. La función `usedQubits` recorre recursivamente el AST para identificar todos los qubits involucrados en una sub-expresión.

```
usedQubits :: Gate -> [Int]
usedQubits (H q) = [q]
usedQubits (X q) = [q]
usedQubits (CNOT c t) = [c, t]
usedQubits (CCNOT c1 c2 t) = [c1, c2, t]
usedQubits (Seq g1 g2) = usedQubits g1 ++ usedQubits g2
usedQubits (Par g1 g2) = usedQubits g1 ++ usedQubits g2
```

Esta lista se utiliza luego para verificar si la intersección de qubits entre dos ramas paralelas es vacía mediante la función `intersect`. Si hay solapamiento, el simulador lanza un error de `InvalidCircuit`, deteniendo la ejecución antes de realizar cálculos costosos.

## 4.7. Denotación matemática de compuertas (`gateToMatrix`).

La función `gateToMatrix` es la encargada de asignar una **semántica denotacional** al `AST`. Su rol es transformar cada constructor de `Gate` en una única matriz de transformación global de $2^n \times 2^n$.

- **Compuertas Atómicas**: Para compuertas como $H$, $X$ o CNOT, se valida el rango de los qubits involucrados y se construye la matriz correspondiente expandida al espacio de $n$ qubits.
- **Composición Secuencial (`Seq`)**: Representa la ejecución temporal donde una compuerta $g_1$ es seguida por $g_2$. En términos de álgebra lineal, esto se traduce en el producto de matrices aplicado de derecha a izquierda: $M_2 \times M_1$.
- **Composición Paralela (`Par`)**: Representa la aplicación simultánea de compuertas sobre qubits distintos. En nuestro simulador, como las matrices ya operan sobre el espacio global del circuito, si las compuertas actúan sobre qubits disjuntos, sus matrices asociadas conmutan. Por lo tanto, el paralelismo se implementa como una multiplicación matricial validada.

```
gateToMatrix n (Par g1 g2) = do
    let qs1 = usedQubits g1
    let qs2 = usedQubits g2
    if null (intersect qs1 qs2)
        then do
            m1 <- gateToMatrix n g1
            m2 <- gateToMatrix n g2
            return (mmult m2 m1)
        else throwQError (InvalidCircuit "Compuertas paralelas
           deben usar qubits disjuntos")
```

*Lógica de evaluación para el combinador paralelo.*

## 4.8. Ejecución y flujo principal: La función `eval`.

Finalmente, el proceso de simulación une la construcción matricial con la evolución dinámica del estado inicial.

### 4.8.1. Aplicación del circuito (`applyCircuit`).

Es la función que actúa como puente entre la descripción estática (matrices) y la evolución dinámica de la mónada. Su flujo es el siguiente: obtiene la matriz global del circuito, recupera el vector de estado actual (`getState`), realiza la multiplicación matriz-vector y actualiza el estado global con el nuevo vector resultante (`putState`).

```
applyCircuit :: Int -> Gate -> QState ()
applyCircuit n g = do
    mat <- gateToMatrix n g
    st <- getState
    putState (mvecmult mat st)
```

### 4.8.2. Punto de entrada: `eval`.

La función `eval` es el componente de más alto nivel del evaluador. Orquesta el inicio de la simulación realizando las siguientes tareas:

1. **Control de Escala**: Verifica que la dimensión del problema sea manejable (limite de 20 qubits).
2. **Inicialización y Ejecución**: Invoca a `runQState` para ejecutar la función `applyCircuit` comenzando con el estado base $|00\ldots0\rangle$.
3. **Manejo de Resultados**: Desempaqueta el resultado del tipo `Either`, permitiendo que los errores ocurridos en cualquier etapa fluyan hacia la interfaz de usuario de forma controlada.

```
eval :: Circuit -> Either QError (StateVector, QTrace)
eval (Circuit n gs)
    | n > 20 = Left (TooManyQubits n)
    | otherwise = case runQState (applyCircuit n gs) (
       initialState n) of
        Left err -> Left err
        Right (_, finalState, traza) -> Right (finalState, traza)
```

*Implementación del punto de entrada principal del evaluador.*

# 5. Módulo `PrettyPrinter.hs`.

El módulo `PrettyPrinter` cumple la función de *traductor* entre las estructuras de datos internas de Haskell y la notación científica estándar utilizada por los humanos en computación cuántica. Su objetivo es que el resultado del simulador no sea una simple lista de números complejos, sino una expresión clara en **notación de Dirac**.

## 5.1. Conversión a binario (`intToBinary`).

Como el `StateVector` es una lista indexada de forma decimal, el primer paso para mostrar un estado como $|101\rangle$ es convertir el índice de la lista a su representación binaria. La función `intToBinary` realiza esta tarea asegurando que la longitud de la cadena coincida con el número total de qubits ($n$):

```
intToBinary :: Int -> Int -> String
intToBinary n i = [ if testBit i k then '1' else '0' | k <- [n-1,
    n-2 .. 0]]
```

Es importante notar que el recorrido de los bits se realiza desde $n-1$ hasta 0, lo cual es consistente con la convención de bits menos significativos (LSB) adoptada en todo el proyecto.

## 5.2. Formateo de amplitudes (`formatComplex`).

En mecánica cuántica, es común trabajar con números complejos cuyas amplitudes reales o imaginarias pueden ser nulas o negativas. La función `formatComplex` limpia la salida para mejorar la legibilidad:

- **Simplificación**: Oculta las partes de la amplitud que son menores a $10^{-10}$ (consideradas cero por errores de coma flotante).
- **Estética**: Si la amplitud es 1,0, se omite el escalar; si es $-1{,}0$, se muestra simplemente un signo menos.
- **Precisión**: Utiliza formato de dos decimales para las amplitudes que no son números enteros.

```
formatComplex :: Complex Double -> String
formatComplex (re :+ im)
    | abs im < 1e-10 && abs (re - 1) < 1e-10 = ""
    | abs im < 1e-10 && abs (re + 1) < 1e-10 = "-"
    | abs im < 1e-10 = printf "%.2f" re
    | abs re < 1e-10 = printf "%.2fi" im
    | im > 0 = printf "(%.2f + %.2fi)" re im
    | otherwise = printf "(%.2f - %.2fi)" re (abs im)
```

## 5.3. Representación de Dirac (`prettyPrint`).

La función principal `prettyPrint` orquestra la transformación del vector de estado. Su lógica se basa en el principio de que solo nos interesan los estados que tienen una proba-

bilidad de ocurrencia mayor a cero.

```
prettyPrint :: Int -> StateVector -> String
prettyPrint n sv =
    let indicesEstados = zip [0..] sv
        estadosActivos = filter (\(_, v) -> magnitude v > 1e-10)
           indicesEstados
        formatTerm (i, v) = formatComplex v ++ "|" ++ intToBinary
            n i ++ ">"
        terms = map formatTerm estadosActivos
    in if null terms then "|0>" else intercalate " + " terms
```

*Implementación de la notación de Dirac.*

El proceso sigue estos pasos:

1. **Zip**: Asocia cada amplitud compleja con su índice decimal.
2. **Filter**: Elimina los estados cuya magnitud es despreciable, limpiando el resultado de ramas con probabilidad 0.
3. **Map**: Convierte cada par activo en una cadena con el formato $c_i\,|binary\rangle$.
4. **Intercalate**: Une todos los términos con el símbolo "+".

Gracias a este módulo, un estado de Bell generado internamente como una lista de complejos se presenta al usuario de forma elegante: `0.71|00>+ 0.71|11>`.

# 6. Compilación y guía de uso.

Para garantizar la reproducibilidad y el correcto funcionamiento del simulador en cualquier plataforma, el proyecto utiliza **Stack** como herramienta de gestión de construcción. Stack abstrae la instalación del compilador GHC y la descarga de dependencias, asegurando un entorno aislado y consistente.

## 6.1. Compilación del sistema.

Para compilar el proyecto por primera vez, el usuario debe situarse en la raíz del directorio `tp-final-alp/` y ejecutar el comando:

```
stack build
```

Este comando realizará las siguientes tareas automáticamente:

1. Descargará la versión exacta del compilador GHC especificada en la configuración.
2. Resolverá e instalará las dependencias necesarias (especialmente la librería `Parsec` para el procesamiento de archivos `.q`).
3. Compilará los módulos del núcleo (`src/`) y generará el ejecutable binario para su sistema operativo.

## 6.2. Ejecución de circuitos.

### 6.2.1. Ejecución por defecto.

Si se ejecuta el comando sin argumentos adicionales, el programa correrá una serie de circuitos de ejemplo embebidos en el código (como el estado de Bell o el circuito GHZ). Esto es útil para verificar que la instalación es correcta:

```
stack run
```

### 6.2.2. Ejecución de archivos específicos.

Para simular un circuito definido en un archivo externo con extensión `.q`, simplemente se pasa la ruta del archivo como argumento. El sistema leerá el archivo, validará su sintaxis y procederá con la simulación:

```
stack run examples-q/bell.q
```

En ambos casos, el simulador imprime un log detallado (la traza) que indica qué compuertas se aplicaron exitosamente y en qué orden, seguido de la representación en notación de Dirac del estado final.

# 7. Ejemplos y Resultados.

En esta sección se presentan los resultados de ejecutar el simulador sobre diversos circuitos de prueba. Estos ejemplos han sido seleccionados para demostrar la capacidad del lenguaje de manejar entrelazamiento, superposición y lógica computacional compleja.

## 7.1. Estado de Bell (`bell.q`).

El **circuito de Bell** genera el nivel más simple de entrelazamiento cuántico entre dos qubits. Es el ejemplo fundamental para verificar que el simulador maneja correctamente la correlación de estados.

**Descripción:** Se aplica una compuerta Hadamard al qubit 1 para ponerlo en superposición ($\frac{|0\rangle+|1\rangle}{\sqrt{2}}$), y luego una CNOT controlada por este para invertir el qubit 0 únicamente si el primero es 1.

```
qubits 2
H 1 ; CNOT 1 0
```

*Código de bell.q.*

**Resultado:**

```
$ stack run examples-q/bell.q

Circuito : Circuito[2 qubits]: H(1) ; CNOT(1,0)
Pasos ejecutados:
  - Hadamard aplicada al qubit 1
  - CNOT: control=1, objetivo=0
Estado final: 0.71|00> + 0.71|11>
```

## 7.2. Estado GHZ de 3 qubits (`ghz.q`).

El **estado Greenberger-Horne-Zeilinger (GHZ)** es una generalización del estado de Bell para tres o más qubits. Es una pieza clave en protocolos de comunicación cuántica y corrección de errores.

**Descripción:** El circuito encadena dos compuertas CNOT tras una Hadamard inicial. El qubit 2 actúa como control del 1, y este a su vez actúa como control del 0. Esto genera una cadena de dependencia que entrelaza a los tres miembros del sistema.

```
qubits 3
H 2 ; CNOT 2 1 ; CNOT 1 0
```

*Código de ghz.q.*

**Resultado:**

```
$ stack run examples-q/ghz.q

Circuito : Circuito[3 qubits]: H(2) ; CNOT(2,1) ; CNOT(1,0)
Pasos ejecutados:
  - Hadamard aplicada al qubit 2
  - CNOT: control=2, objetivo=1
  - CNOT: control=1, objetivo=0
Estado final: 0.71|000> + 0.71|111>
```

## 7.3. Algoritmo de Búsqueda de Grover (`grover.q`).

El **Algoritmo de Grover** se utiliza para buscar un elemento específico dentro de una base de datos no estructurada. Mientras que un algoritmo clásico requiere $\mathcal{O}(N)$ intentos para encontrar un elemento entre $N$ posibilidades, Grover lo logra en $\mathcal{O}(\sqrt{N})$ pasos.

### 7.3.1. Propósito y fundamento.

El objetivo es encontrar un estado base 'marcado' (al que llamaremos $w$). El algoritmo no mira los elementos uno por uno, sino que utiliza la **interferencia cuántica** para amplificar la amplitud de probabilidad del estado correcto y reducir la de los estados incorrectos. El proceso se compone de dos etapas principales que se repiten:

1. **El Oráculo**: Invierte la fase del estado marcado ($|w\rangle \to -|w\rangle$). Es como ponerle una 'marca' física al estado que buscamos sin alterar a los demás.
2. **Operador de Difusión**: Realiza una inversión respecto a la media de todas las amplitudes. Dado que el estado marcado ahora tiene una amplitud negativa, esta operación lo 'empuja' hacia arriba, aumentando significativamente su probabilidad.

### 7.3.2. Implementación en 2 qubits.

En un sistema de 2 qubits tenemos 4 estados posibles ($|00\rangle, |01\rangle, |10\rangle, |11\rangle$). En este ejemplo, buscaremos el estado $|11\rangle$.

```
qubits 2
// 1. Preparacion: Superposicion uniforme en todos los estados
H 0 || H 1 ;

// 2. Oraculo para |11>: Invierte la fase si ambos son 1
H 1 ; CNOT 0 1 ; H 1 ;

// 3. Difusion (Inversion sobre la media)
H 0 || H 1 ;
X 0 || X 1 ;
H 1 ; CNOT 0 1 ; H 1 ; // CNOT controlada para fase
X 0 || X 1 ;
H 0 || H 1
```

### 7.3.3. Resultado.

Al ejecutar este circuito, el simulador muestra cómo el sistema, que inicialmente tenía un 25 % de probabilidad para cada estado, concentra casi toda su masa de probabilidad en el estado buscado.

```
$ stack run examples-q/grover.q

Circuito : Circuito[2 qubits]: (H(0) || H(1)) ; H(1) ; CNOT(0,1)
   ; H(1) ; (H(0) || H(1)) ; (X(0) || X(1)) ; H(1) ; CNOT(0,1) ; H
   (1) ; (X(0) || X(1)) ; (H(0) || H(1))
Pasos ejecutados:
  - Hadamard aplicada al qubit 0
  - Hadamard aplicada al qubit 1
  - Hadamard aplicada al qubit 1
  - CNOT: control=0, objetivo=1
  - Hadamard aplicada al qubit 1
  - Hadamard aplicada al qubit 0
  - Hadamard aplicada al qubit 1
  - Compuerta X (NOT) aplicada al qubit 0
  - Compuerta X (NOT) aplicada al qubit 1
  - Hadamard aplicada al qubit 1
  - CNOT: control=0, objetivo=1
  - Hadamard aplicada al qubit 1
  - Compuerta X (NOT) aplicada al qubit 0
  - Compuerta X (NOT) aplicada al qubit 1
  - Hadamard aplicada al qubit 0
  - Hadamard aplicada al qubit 1
Estado final: -|11>
```

**Interpretación**: Gracias a que en 2 qubits una sola iteración de Grover es suficiente para alcanzar una probabilidad del 100 %, el simulador nos devuelve el estado $|11\rangle$ puro.

## 7.4. Medio sumador cuántico (`adder.q`).

Este ejemplo implementa un **Half-Adder** funcional utilizando 4 qubits, demostrando cómo la computación cuántica puede emular lógica clásica mediante operaciones reversibles.

**Descripción:** El circuito utiliza los qubits 3 y 2 como entradas, el qubit 1 para el *Acarreo* (Carry) y el 0 para la *Suma* (Sum).

- **Acarreo**: Se calcula mediante una compuerta **CCNOT** (Toffoli). El acarreo es 1 solo si ambos qubits de entrada son 1 (equivalente a una compuerta AND clásica).
- **Suma**: Se calcula aplicando dos compuertas **CNOT** sobre el qubit objetivo 0. Esto equivale a una operación XOR clásica.

```
qubits 4
X 3 ; X 2          // Inicializamos entradas en |11> (1 + 1)
CCNOT 3 2 1       // Acarreo: Q3 AND Q2 -> Q1
CNOT 3 0          // Suma: Q3 XOR Q2 -> Q0
CNOT 2 0
```

*Código de adder.q (1+1).*

**Resultado:**

```
$ stack run examples-q/adder.q

Circuito : Circuito[4 qubits]: X(3) ; X(2) ; CCNOT(3,2,1) ; CNOT
   (3,0) ; CNOT(2,0)
Pasos ejecutados:
  - Compuerta X (NOT) aplicada al qubit 3
  - Compuerta X (NOT) aplicada al qubit 2
  - CCNOT: control=3, control=2, objetivo=1
  - CNOT: control=3, objetivo=0
  - CNOT: control=2, objetivo=0
Estado final: |1110>
```

**Análisis del resultado:** En el estado final $|1110\rangle$:

- Los bits de la izquierda corresponden a las entradas (1 y 1).
- El bit en la posición del acarreo (qubit 1) es **1**.
- El bit en la posición de la suma (qubit 0) es **0**.
- El resultado binario es $10_2$, que es efectivamente $2_{10}$ $(1 + 1)$.

# 8. Decisiones de Diseño y Justificaciones.

El desarrollo del simulador ha implicado una serie de decisiones técnicas orientadas a la robustez, la fidelidad física y la eficiencia computacional.

- ***Deep Embedding* y GADTs**: Se optó por un *deep embedding* utilizando **GADTs** para representar el AST. Esta decisión permite tratar a los circuitos no solo como funciones de ejecución, sino como estructuras de datos que pueden ser analizadas, optimizadas o impresas (como se hace en el módulo `PrettyPrinter`). El uso de GADTs garantiza seguridad de tipos y facilita el reconocimiento de patrones en el evaluador.
- **Convención LSB (*Least Significant Bit*)**: En este proyecto, el qubit 0 se considera el bit menos significativo ($2^0$). Esta convención es estándar en gran parte de la literatura y simplifica la implementación de la manipulación de bits en las funciones `expandCNOT` y `expandCCNOT`.
- **Uso de `Complex Double`**: La computación cuántica requiere una representación precisa de las fases y amplitudes. El tipo `Complex Double` de Haskell proporciona la precisión necesaria para evitar que errores acumulados de coma flotante desvíen los resultados en algoritmos con muchas iteraciones (como el de Grover).
- **Evitación de SWAPs**: Como se fundamentó en la sección de álgebra lineal, se decidió construir las matrices de compuertas controladas de forma directa mediante lógica de bits. Esto mejora el rendimiento de $\mathcal{O}(N^3)$ a $\mathcal{O}(N^2)$ y hace que el núcleo del simulador sea mucho más limpio y menos propenso a errores de reordenamiento de hilos.

# 9. Posibles extensiones futuras.

Si bien el simulador es completamente funcional para los objetivos planteados, el diseño modular del EDSL permite considerar diversas ampliaciones para convertirlo en una herramienta aún más potente:

1. **Inclusión de nuevas compuertas**: El sistema podría extenderse fácilmente para soportar un conjunto de compuertas más amplio, como las compuertas de fase $(S, T)$, rotaciones arbitrarias $(R_x, R_y, R_z)$ o la compuerta SWAP, permitiendo simular una variedad mayor de algoritmos.
2. **Optimización de Circuitos**: Desarrollar un paso de pre-procesamiento que analice el AST para simplificar operaciones antes de la evaluación matricial. Por ejemplo, detectar secuencias de compuertas que se anulan entre sí, como dos compuertas de Hadamard seguidas ($H \cdot H = I$), reduciendo significativamente la carga computacional en circuitos extensos.
3. **Algoritmo de Shor**: Con la inclusión de compuertas de fase y la Transformada Cuántica de Fourier (QFT), el simulador sería capaz de ejecutar el Algoritmo de Shor para la factorización de números enteros en tiempo polinomial, demostrando uno de los usos más disruptivos de la computación cuántica.

# 10. Conclusiones.

Este trabajo práctico demuestra la idoneidad de Haskell como herramienta para el diseño de **Lenguajes de Dominio Específico** en ámbitos científicos. Las características principales que hacen a Haskell ideal para este problema son:

- **Sistema de tipos fuerte**: Los GADTs garantizan que el AST sólo represente circuitos bien formados.
- **Abstracción monádica**: La mónada `QState` encapsula limpiamente el estado cuántico, el manejo de errores y la traza de ejecución.
- **Paradigma funcional**: Las funciones puras (sin efectos secundarios) para operaciones matriciales facilitan el razonamiento formal sobre su correctitud.
- **Ecosistema (Parsec, Stack)**: Las librerías estándar de Haskell proveen combinadores de parsers y gestión de proyectos de nivel profesional.

En conclusión, se ha desarrollado un simulador robusto, extensible y con una sintaxis amigable que abstrae la complejidad del álgebra lineal subyacente, permitiendo al usuario centrarse en el diseño de algoritmos cuánticos.

# Referencias

[1] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.

[2] Apuntes de la cátedra de Análisis de Lenguajes de Programación (ALP). *Facultad de Ciencias Exactas, Ingeniería y Agrimensura (FCEIA) - Universidad Nacional de Rosario (UNR).*